Описание робота-балансира

Двухколесный робот-балансир — это самобалансирующийся двухколесный робот(Рис.13). В процессе моделирования будем использовать объектно-ориентированный способ проектирования робота.

Модельно-ориентированное проектирование — это метод разработки программного обеспечения, в котором используются имитационные модели. Обычно его обозначают аббревиатурой MBD. Для систем управления проектировщик моделирует установку и контроллер или их часть и тестирует алгоритм контроллера на основе компьютерного моделирования или моделирования в реальном времени. Моделирование в реальном времени позволяет нам проверять и проверять алгоритм в реальном времени, используя код, сгенерированный из модели.

Робот оснащен следующими датчиками:

1) Энкодером, установленным на валу двигателя, возвращающие значения угла поворота в градусах (значение записываются в переменную в целых числах int32)
2) Ультразвуковым датчиком расстояния
3) Гироскопом для отслеживания угла отклонения робота от вертикали

Вообще говоря, датчики и исполнительные механизмы индивидуальны. В частности, нужно иметь в виду, что смещение гироскопа и дрейф гироскопа имеют большое влияние на контроль баланса. Смещение гироскопа является выходным сигналом, когда датчик гироскопа не вращается, а дрейф гироскопа представляет собой изменение смещения гироскопа во времени.

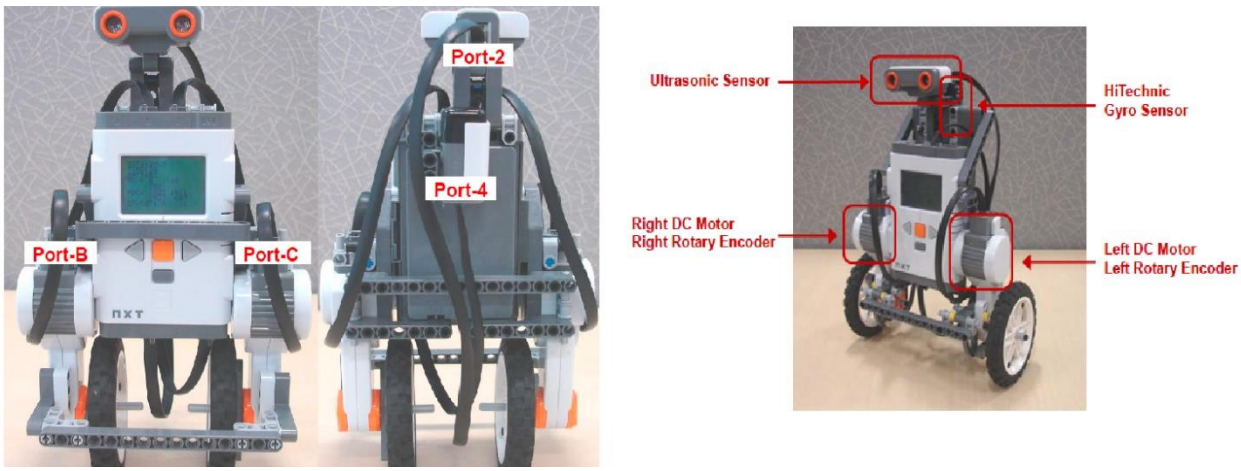


Рис.13 Внешний вид робота-балансира

Математическое описание робота

Роботы-балансиры можно рассматривать как двухколесную модель перевернутого маятника(Рис.14)

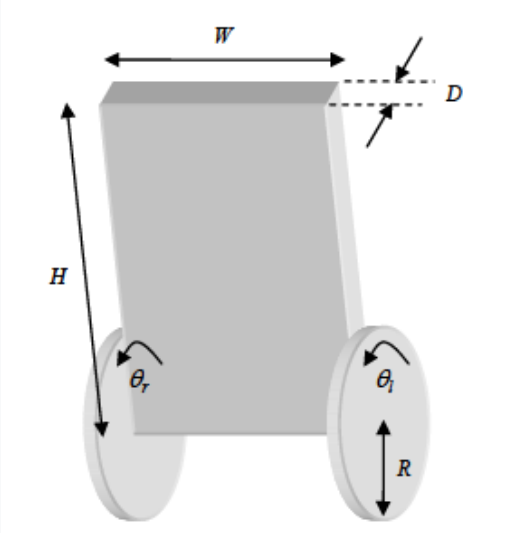


Рис.14. Модель робота-балансира

Модель управления роботом-балансиром представим как перевернуты маятники и представим видом сбоку и видом сверху(Рис.15). На рисунке буковой ψ обозначен угол тангажа, $\theta_{r,l}$-угол поворота правого и левого колеса робота, $\theta_{m_{r,l}}$-угол поворота двигателей робота. А так же представим робота в декартовой системе координат и выбурим начало отчета.

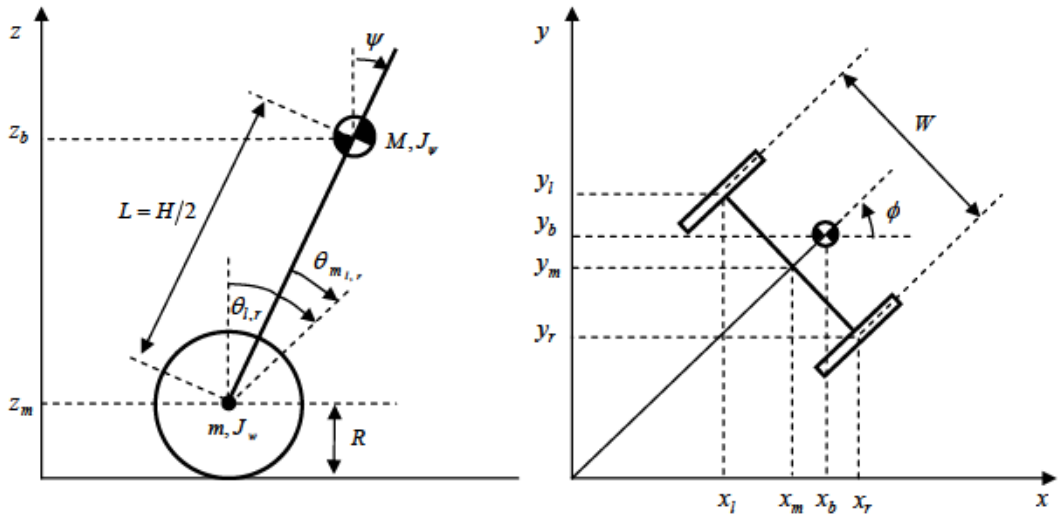


Рис.15 Вид сбоку и вид сверху на двухколесный перевернутый маятник

Для определения место положения робота введём средний угол поворота колес Θ и угол направления движения ϕ:

$$\theta = \frac{1}{2}(\theta_l + \theta_r) \qquad 6$$

$$\phi = \frac{R}{W}(\theta_l - \theta_r)$$

Уравнения движения двухколесного перевернутого маятника

Выведем согласно рис.15 уравнения движения двухколесного перевернутого маятника методом Лагранжа на основе системы координат.

$$(\theta, \phi) = \left(\frac{1}{2}(\theta_l + \theta_r)\ \frac{R}{W}(\theta_l - \theta_r)\right) \qquad 5$$

$$(x_m,\ y_m,\ z_m,) = (\int \dot{x_m} dt\ ,\ \int \dot{y_m,} dt\ , R,\ (\dot{x}_m,\ \dot{y}_m) = (R\dot{\theta} cos(\phi)\ \ R\dot{\theta} sin(\phi)) \qquad 6$$

$$(x_l\ \ y_l\ \ z_l) = (x_m - \frac{W}{2} sin(\phi),\ y_m + \frac{W}{2} cos(\phi),\ z_m) \qquad 7$$

$$(x_r\ \ y_r\ \ z_r) = (x_m + \frac{W}{2} sin(\phi),\ y_m - \frac{W}{2} cos(\phi),\ z_m) \qquad 8$$

$$(x_b\ \ y_b\ \ z_b) = (x_m + Lsin(\psi)cos(\psi),\ y_m + Lsin(\psi)cos(\psi),\ z_m + Lcos(\psi)) \qquad 8$$

Поступательная кинетическая энергия T_1, вращательная кинетическая энергия T_2, потенциальная энергия U:

$$T_1 = \frac{1}{2}m(\dot{x}_l^2 + \dot{y}_l^2 + \dot{z}_l^2) + \frac{1}{2}m(\dot{x}_r^2 + \dot{y}_r^2 + \dot{z}_r^2) + \frac{1}{2}M(\dot{x}_b^2 + \dot{y}_b^2 + \dot{z}_b^2) \qquad 9$$

$$T_1 = \frac{1}{2}J_w\dot{\theta}_l^2 + \frac{1}{2}J_w\dot{\theta}_r^2 + \frac{1}{2}J_\psi\dot{\psi}_l^2 + \frac{1}{2}J_{\phi 1}\dot{\phi}_l^2 + \frac{1}{2}n^2J_m(\dot{\theta}_l - \dot{\psi})^2 + \frac{1}{2}n^2J_m(\dot{\theta}_r - \dot{\psi})^2 \qquad 10$$

Пятые и шестые члены уравнения 10 – это кинетическая энергия вращения левого и правого двигателя постоянного тока могут быть представлены:

$$L = T_1 + T_2 - U \qquad 11$$

В качестве обобщенных координат мы используем следующие переменные:

Θ-Средний угол левого и правого колеса

Ψ-Угол тангажа кузова

ϕ- Угол рыскания кузова

Уравнения Лагранжа для нашего робота:

$$\frac{d}{dt}(\frac{\partial L}{\partial \dot{\theta}}) - (\frac{\partial L}{\partial \theta}) = F_\theta \qquad 11$$

$$\frac{d}{dt}(\frac{\partial L}{\partial \dot{\psi}}) - (\frac{\partial L}{\partial \psi}) = F_\psi \qquad 12$$

$$\frac{d}{dt}(\frac{\partial L}{\partial \dot{\phi}}) - (\frac{\partial L}{\partial \phi}) = F_\phi \qquad 13$$

Путем оценки уравнений 11-13 мы получаем следующие уравнения:

$$[(2m+M)R^2+2J_w+2n^2J_m]\ddot{\theta}+(MLRcos(\psi)-2n^2J_m)\ddot{\psi}-MLR\dot{\psi}^2sin(\psi)=F_\theta \quad 14$$

$$(MLRcos(\psi)-2n^2J_m)\ddot{\theta}+(ML^2+J_\psi+2n^2J_m)\ddot{\psi}-MgLsin(\psi)-ML^2\phi^2sin(\psi)cos(\psi)=F_\psi \quad 15$$

$$\left[\frac{1}{2}mW^2+J_\phi+\frac{W^2}{2R^2}(J_w+n^2J_m)+ML^2sin^2(\psi)\right]\ddot{\phi}+2ML^2\dot{\psi}\dot{\phi}sin(\psi)cos(\psi)=F_\phi \quad 16$$

В левой части описана динамика движения робота, его инерция, а в правой части внешние моменты силы, действующие на него по осям Θ, ψ, ϕ.

Опишем внешние силы, действующие на робота:

Момент силы тяжести, который стремится его уронить:

$$F_g=Mgl\theta \qquad 17$$

Со стороны двигателя в свою очередь действуют следующие силы:

Момент, создаваемый на валу двигателя, который пропорционален величине питающего напряжения и два демпфирующих момента (момент противо-ЭДС, и момент вязкого трения внутри двигателя):

$$\mathrm{F_m}=\alpha \mathrm{v}-\beta\dot{\theta}_\mathrm{m} \quad 18$$

$$\alpha=\mathrm{n}\frac{\mathrm{K_t}}{\mathrm{R_m}} \qquad 19$$

$$\beta=\mathrm{n}\frac{\mathrm{K_tK_b}}{\mathrm{R_m}}+\mathrm{f_m} \qquad 20$$

Со стороны пола действует сила трения, пропорциональный скорости:

$$\mathrm{F_w}=-\mathrm{f_w}\dot{\theta} \qquad 21$$

Просуммируем их:

$$F_\theta=F_{m_r}+F_{m_l}+F_{w_r}+F_{w_l} \quad 22$$

$$F_\psi=-F_{m_l}-F_{m_l}+F_g \qquad 23$$

$$F_\phi=\frac{W}{2R}(F_{m_r}-F_{m_l}+F_{w_r}-F_{w_l}) \qquad 24$$

Уравнения состояния двухколесного перевернутого маятника

Выведем уравнения состояния на основе теории управления путем линеаризации уравнений движения в точке баланса робота-балансира, путем рассмотрения предел ψ→0 (sin(ψ) →0, cos(ψ)→1) и пренебрегаем членом второго порядка типа $\dot{\psi}^2$. Уравнения движения (14) – (16) аппроксимируются следующим образом:

$$[(2m+M)R^2+2J_w+2n^2J_m]\ddot{\theta}+(MLR-2n^2J_m)\ddot{\psi}=F_\theta \qquad 25$$

$$(MLRc-2n^2J_m)\ddot{\theta}+(ML^2+J_\psi+2n^2J_m)\ddot{\psi}-MgL\psi=F_\psi \qquad 26$$

$$\left[\frac{1}{2}mW^2+J_\phi+\frac{W^2}{2R^2}(J_w+n^2J_m)\right]\ddot{\phi}=F_\phi \qquad 27$$

Уравнение 25 и уравнение 26 имеет Θ и ϕ, а уравнение 27 имеет только ϕ. Эти уравнения можно представить в матричной форме:

$$\mathrm{E}\begin{bmatrix}\ddot{\theta}\\ \ddot{\psi}\end{bmatrix}+\mathrm{F}\begin{bmatrix}\dot{\theta}\\ \dot{\psi}\end{bmatrix}+\mathrm{G}\begin{bmatrix}\theta\\ \psi\end{bmatrix}=\mathrm{H}\begin{bmatrix}\mathrm{v_l}\\ \mathrm{v_r}\end{bmatrix} \quad 28$$

, где E, F,G,H – матричные коэффициенты линеаризованной модели:

$$\mathrm{E}=\begin{bmatrix}(2\mathrm{m}+\mathrm{M})\mathrm{R}^2+2\mathrm{J_w}+2\mathrm{n}^2\mathrm{J_m} & \mathrm{MLR}-2\mathrm{n}^2\mathrm{J_m}\\ \mathrm{MLR}-2\mathrm{n}^2\mathrm{J_m} & \mathrm{ML}^2+\mathrm{J_\psi}+2\mathrm{n}^2\mathrm{J_m}\end{bmatrix}$$

$$\mathrm{F}=\mathrm{s}\begin{bmatrix}\beta+\mathrm{f_w} & -\beta\\ -\beta & \beta\end{bmatrix}$$

$$\mathrm{G}=\begin{bmatrix}0 & 0\\ 0 & -\mathrm{MGL}\end{bmatrix}$$

$$\mathrm{H}=\begin{bmatrix}\alpha & \alpha\\ -\alpha & \alpha\end{bmatrix}$$

Уравнение 27 запишем в виде:

$$\mathrm{I}\ddot{\phi}+\mathrm{J}\dot{\phi}=\mathrm{K}(\mathrm{v_r}-\mathrm{v_l}) \quad 29$$

, где

$$I = \frac{1}{2} m W^2 + J_\phi + \frac{W^2}{2R^2}(J_w + n^2 J_m)$$

$$J = \frac{W^2}{2R^2}(\beta + f_w)$$

$$K = \frac{W}{2R}\alpha$$

Метод пространство состояния

Пространство состояний обычно называют фазовым пространством динамической системы, а траекторию движения изображающей точки в этом пространстве — фазовой траекторией.

В пространстве состояний создаётся модель динамической системы, включающая набор переменных входа, выхода и состояния, связанных между собой дифференциальными уравнениями первого порядка, которые записываются в матричной форме. В отличие от описания в виде передаточной функции и других методов частотной области, пространство состояний позволяет работать не только с линейными системами и нулевыми начальными условиями.

В основе метода лежит - представление реальной исследуемой системы в виде многомерного (векторного) объекта. В отличие от скалярного (имеющего один вход и один выход) у векторного объекта (или системы) может быть несколько входных и несколько выходных переменных (сигналов, координат).

К входным переменным, например, относятся материальные потоки, поступающие от блока дозирующих устройств, режимные параметры дозаторов, от которых зависит структура этих потоков, динамика подачи материалов от дозаторов к смесительному устройству и т.д.

К выходным управляемым переменным относятся показатели качества результирующей смеси (содержания основного компонента и сопутствующих добавок), а также большое число режимноконструктивных и расходовых параметров в разных точках смесительного устройства (в каналах прямой и обратной подачи материалопотоков).

В качестве выходных переменных могут использоваться как реальные, поддающиеся измерению, физические переменные, так и абстрактные, не поддающиеся измерению, переменные, например, производные (скорости, ускорения, импульсы и т.д.) от наблюдаемых выходных переменных. На такой основе любой скалярный динамический элемент, описываемый дифференциальным уравнением *n-го* порядка (при $n > 1$), может рассматриваться как многомерный (векторный).

Таким образом, с помощью определенного набора переменных (входных, выходных, внутренних) можно полностью охарактеризовать *состояние* любой динамической (векторной или скалярной) системы.

Математическая модель динамики системы, оперирующая переменными состояния, является моделью, сформированной в так называемом **пространстве состояний.** Данная модель при этом записывается через функции-оригиналы и не использует изображения по Лапласу. Модель записывается частично в виде дифференциального уравнения 1-го порядка в векторноматричной форме (ВМФ), частично - в виде векторноматричного алгебраического уравнения; в структурном отношении дифференциальные уравнения (ДУ) должны иметь так называемую нормальную *форму Коши*, когда в левой части ДУ записывается первая производная некоторой переменных состояния с коэффициентом «1», а в правой - все остальные члены ДУ.

Полная математическая модель линейной векторной (многомерной) динамической системы n-го порядка состоит из двух векторно-матричных уравнений:

- - дифференциального *уравнения состояния* первого порядка (в нормальной форме Коши);

- - алгебраического *уравнения выхода* (наблюдения).

В скалярной форме матричное уравнение состояния записывается в виде *n* обыкновенных дифференциальных уравнений 1-го порядка.

Для одномерного объекта (системы) *n*-го порядка переменными состояния могут служить выходная координата и ее (n-1) производных. В этом случае, т.е. когда переменные состояния связаны соотношением:

$\frac{dx_i(t)}{dt} = \dot{x}_j(t) = x_{(j+1)}(t)$ 30

, где $j = \overline{1,\ n-1}$

переменные состояния называются ***фазовыми переменными.*** При этом *n* - мерное пространство, координаты которого - переменные состояния, именуется *пространством состояния*, а способ описания систем посредством таких переменных называется ***методом пространства состояний (методом переменных состояния).***

Таким образом, **состояние системы описывается вектором переменных состояния (ПС):**

$$\overline{x_j(t)}, \qquad j = \overline{1,n}$$

где n - порядок характеристического полинома.

Вектор $\overline{x_j(t)}$, пространство состояний, изменяясь во времени в гиперпространстве *R"* состояния системы, образует в нем *гиперповерхность состояния.* Помимо , пространство состояний, на выходе фиксируется вектор $y_k(t)$ выходных координат (или иначе - вектор наблюдения), а на входе действует вектор $u_f(t)$ управляющих воздействий (вектор управления).

Математическая модель динамической системы (ДС) в терминах пространства состояний запишется так:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$ 31

, где A-матрица системы. Характеризует ее структуру, вязкость, динамику, жесткость и другие параметры; B- матрица управления. Описывает реакцию системы на управление; C-матрица выхода. Характеризует выход; D-матрица прямой связи. Показывает на сколько управление передается на выход миную систему.

$x = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}$ -Вектор состояния. Он включает в себя все переменные, которые описывают состояние системы в любой момент времени.

$u = \begin{bmatrix} u_1 \\ u_2 \\ \dots \\ u_n \end{bmatrix}$ -Вектор управления. Включает в себя компоненты, при помощи которых мы управляем системой.

$y = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix}$ -Вектор выхода. Значения выходных параметров.

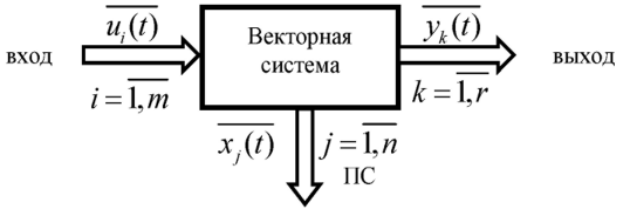


Рис.16 Векторная система

Блок-схема реализации математической модели представлена на рис. 17

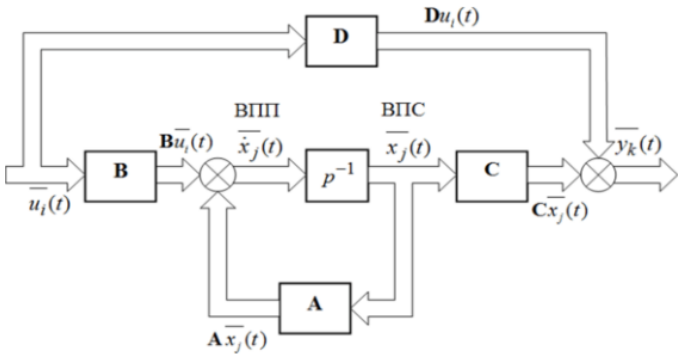


Рис.17 Блок-схема структурной реализации векторно-матричной математической модели
Здесь ВПС - вектор переменных состояния; ВПС - вектор производных переменных состояния; p^{-1} - интегратор.

Представление робота балансира методом пространство состояния

Преобразование начнем с уравнения 29, оно описывает поворот робота:

$$\mathrm{I}\ddot{\phi} + \mathrm{J}\dot{\phi} = \mathrm{K}(\mathrm{v_r} - \mathrm{v_l})$$

Это дифференциальное уравнение второго порядка. И это означает, что система которую она описывает имеет 2 состояния:

$x_1 = \phi$ - это угол поворота

$x_2 = \dot{\phi}$ - скорость поворота

Управляющие величины:

$u_1 = v_l$ – напряжение на левом двигателе

$u_2 = v_r$ - напряжение на правом двигателе

На выходе нас интересует:

$y_1 = \phi$ – напряжение на левом двигателе

$y_2 = \dot{\phi}$ - напряжение на правом двигателе

Преобразуем уравнение 29:

$$\ddot{\phi} = -\frac{J}{I}\dot{\phi} + \frac{K}{I}(\mathrm{v_r} - \mathrm{v_l}) \quad 30$$

Составим систему дифференциальных уравнений:

$$\begin{cases} \dot{\phi} = \dot{\phi} \\ \ddot{\phi} = -\frac{J}{I}\dot{\phi} + \frac{K}{I}(\mathrm{v_r} - \mathrm{v_l}) \end{cases} \quad 31$$

Произведем замену переменных:

$$\dot{\phi} = \dot{x_1} = x_2$$
$$\ddot{\phi} = \dot{x_2}$$

Переходим к уравнению пространство состояний, добавив уравнение выхода системы:

$$\begin{cases} \dot{x_1} = x_2 \\ \dot{x_2} = -\frac{J}{I}x_2 - \frac{K}{I}(\mathrm{v_r} - \mathrm{v_l}) \\ y_1 = x_1 \\ y_2 = x_2 \end{cases} \quad 32$$

Переведем уравнение к следующему виду, дополнив недостающие переменные нулевыми коэффициентами:

$$\begin{cases} \dot{x_1} = 0x_1 + 1x_2 + 0u_1 + 0u_2 \\ \dot{x_2} = 0x_1 - \frac{J}{I}x_2 - \frac{K}{I}\mathrm{u} + \frac{K}{I}\mathrm{u_2} \\ y_1 = 1x_1 + 0x_1 + 0u_1 + 0u_2 \\ y_2 = 0x_1 + 1x_2 + 0u_1 + 0u_2 \end{cases} \quad 33$$

Из которого матрицы получаются:

$$A=\begin{bmatrix}0 & 1\\ 0 & -\frac{J}{I}\end{bmatrix};\ B=\begin{bmatrix}0 & 0\\ -\frac{K}{I} & \frac{K}{I}\end{bmatrix};\ C=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix};\ D=\begin{bmatrix}0 & 0\\ 0 & 0\end{bmatrix};\ x=\begin{bmatrix}\phi\\ \dot{\phi}\end{bmatrix};\ u=\begin{bmatrix}v_l\\ v_r\end{bmatrix};\ y=\begin{bmatrix}\phi\\ \dot{\phi}\end{bmatrix}$$

Рассмотрим уравнение 28:

$$\mathrm{E}\begin{bmatrix}\ddot{\theta}\\ \ddot{\psi}\end{bmatrix}+\mathrm{F}\begin{bmatrix}\dot{\theta}\\ \dot{\psi}\end{bmatrix}+\mathrm{G}\begin{bmatrix}\theta\\ \psi\end{bmatrix}=\mathrm{H}\begin{bmatrix}\mathrm{v_l}\\ \mathrm{v_r}\end{bmatrix}$$

Оно содержит 2 уравнения второго порядка, поэтому вектор состояния содержит 4 переменные, вектор управления 2 переменные, вектор выхода 4 переменные:

$$x=\begin{bmatrix}\phi\\ \psi\\ \dot{\phi}\\ \dot{\psi}\end{bmatrix};\ u=\begin{bmatrix}v_l\\ v_r\end{bmatrix}\begin{bmatrix}u_1\\ u_2\\ \cdots\\ u_n\end{bmatrix};\ y=\begin{bmatrix}\phi\\ \psi\\ \dot{\phi}\\ \dot{\psi}\end{bmatrix}.$$

Преобразуем уравнение 28, до множив уравнение на обратную матрицу $\mathrm{E^{-1}}$:

$$\begin{bmatrix}\ddot{\theta}\\ \ddot{\psi}\end{bmatrix}=-\mathrm{E^{-1}G}\begin{bmatrix}\theta\\ \psi\end{bmatrix}-\mathrm{E^{-1}F}\begin{bmatrix}\dot{\theta}\\ \dot{\psi}\end{bmatrix}+\mathrm{E^{-1}H}\begin{bmatrix}\mathrm{v_l}\\ \mathrm{v_r}\end{bmatrix}\qquad 34$$

Подставим значения вектора состояния, управления и выхода:

$$\begin{bmatrix}\dot{x}_3\\ \dot{x}_4\end{bmatrix}=-\mathrm{E^{-1}G}\begin{bmatrix}x_1\\ x_2\end{bmatrix}-\mathrm{E^{-1}F}\begin{bmatrix}\dot{x}_3\\ \dot{x}_3\end{bmatrix}+\mathrm{E^{-1}H}\begin{bmatrix}\mathrm{u_1}\\ \mathrm{u_2}\end{bmatrix}\qquad 35$$

Добавим уравнения для $\dot{x}_1$ и $\dot{x}_2$:

$$\begin{bmatrix}\dot{x}_1\\ \dot{x}_2\end{bmatrix}=\begin{bmatrix}0 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix}+\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}\dot{x}_3\\ \dot{x}_3\end{bmatrix}+\begin{bmatrix}0 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}\mathrm{u_1}\\ \mathrm{u_2}\end{bmatrix}\qquad 36$$

В итоге матрицы пространство состояния:

$$A=\begin{bmatrix}0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\\ -\mathrm{E^{-1}} & G & -\mathrm{E^{-1}} & F\end{bmatrix};\quad \mathrm{B}=\begin{bmatrix}0 & 0\\ 0 & 0\\ \mathrm{E^{-1}} & H\end{bmatrix};\quad \mathrm{C}=[1];\quad \mathrm{D}=[0].$$

В результате всех преобразований получаем модель робота-балансира в пространство состояний:

$$\begin{aligned}\dot{x}_1 &= A_1x_1+B_1u_1\\ y_1 &= C_1x_1+D_1u_1\\ \dot{x}_2 &= A_2x_2+B_2u_2\\ y_2 &= C_2x_2+D_2u_2\end{aligned}\qquad \text{4 уравнения}$$

$$x_1=\begin{bmatrix}\phi\\ \psi\\ \dot{\phi}\\ \dot{\psi}\end{bmatrix};\qquad x_2=\begin{bmatrix}\phi\\ \dot{\phi}\end{bmatrix};$$

$$u_1=\begin{bmatrix}v_l\\ v_r\end{bmatrix};\quad u_2=\begin{bmatrix}v_l\\ v_r\end{bmatrix};$$

$$y_1=\begin{bmatrix}\phi\\ \psi\\ \dot{\phi}\\ \dot{\psi}\end{bmatrix};\qquad y_2=\begin{bmatrix}\phi\\ \dot{\phi}\end{bmatrix};$$

2 входа, 6 состояния, 6 выходов

$$A_1=\begin{bmatrix}0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\\ -\mathrm{E^{-1}} & G & -\mathrm{E^{-1}} & F\end{bmatrix};\quad B_1=\begin{bmatrix}0 & 0\\ 0 & 0\\ \mathrm{E^{-1}} & H\end{bmatrix};\quad C_1=\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix};$$

$$D_1=\begin{bmatrix}0 & 0\\ 0 & 0\\ 0 & 0\\ 0 & 0\end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{J}{I} \end{bmatrix}; B_2 = \begin{bmatrix} 0 & 0 \\ -\frac{K}{I} & \frac{K}{I} \end{bmatrix}; C_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; D_2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

8 матриц

Строим модель робота в Matlab Simulink

Перепишем исходные данные робота – балансира:

```
g = 9.81;                 % ускорение силы тяжести [м/с^2 ]
m = 0.03;                 % вес колеса, [кг]
R = 0.04;                 % радиус колеса, [м]
Jw=m*R^2/2;               % момент инерции колеса, [кгм^2]
M = 0.6;                  % масса робота, [кг]
W = 0.14;                 % ширина робота, [м]
D = 0.04;                 % Толщина робота, [м]
h = 0.144;                % Высота робота, [м]
L = h/2;                  % Расстояние от центра масс робота до центра колеса, [кг]
Jpsi=M*L^2/3;             % Момент инерции (при наклоне), [кгм^2]
Jphi=M*(W^2+D^2)/12;      % Момент инерции (при повороте), [кгм^2]
fw=0;                     % Коэффициент вязкого трения между колесом и полом.
fm=0.0022;                % Коэффициент вязкого трения в моторе

% Параметры двигателя

Jm=1e-5;                  % Момент инерции ротора двигателя, [кгм^2]
Rm= 6.69;                 % Сопротивление обмотки двигателя, [Ом]
Kb=0.468;                 % Коэффициент противо-ЭДС, [В*с/рад]
Kt=0.317;                 % Коэффициент передачи по току, [Н*м/А]
n=1;                      % Коэффициент передачи редуктора
```

Откроем новый Script в матлабе и скопируем исходные данные робота в исполняемый файл и сохраним под названием parametrs_Rob.m. В данном файле будут хранится исходные данные робота.
Для описания модели создадим новый Matlab Script и запишем нашу модель, по ранее выведенным уравнениям, в данном скрипте.
Первым делом будем вызывать Script с параметрами робота, путем обращения к имени ранее сохраненного Scripta:

```
% Загрузка параметров
parametrs_Rob;
```

Следующие строки кода будут описывать переменные для модели:

```
% Переменные модели
alpha = n * Kt / Rm;
beta = n * Kt * Kb / Rm + fm;
E = [(2*m+M)*R^2 + 2*Jw + 2*n^2*Jm   M*L*R - 2*n^2*Jm;
     M*L*R - 2*n^2*Jm              M*L^2 + Jpsi + 2*n^2*Jm];
F = 2*[beta+fw   -beta;
       -beta     beta];
G = [0   0;
     0  -M*g*L];
H = [alpha   alpha;
    -alpha  -alpha];
```

```
I = m * W^2 / 2 + Jphi + (Jw + n^2 * Jm) * W^2 / (2 * R^2);
J = W^2 / (2 * R^2) * (beta + fw);
K = W / (2 * R) * alpha;

% Матрица пространство состояния
A1 = [0 0 1 0;
          0 0   0 1;
     -E\G  -E\F];
B1 = [0 0;
            0 0;
            E\H];
C1 = eye(4);
D1 = zeros(4, 2);
A2 = [0   1
            0  -J/I];
B2 = [0    0
           -K/I  K/I];
C2 = eye(2);
D2 = zeros(2);

% Модель пространство состояния
s1 = ss(A1, B1, C1, D1);
s1.StateName = {'theta', 'psi', 'theta_dot', 'psi_dot'};
s1.InputName = {'Vl', 'Vr'};
s1.OutputName = {'theta', 'psi', 'theta_dot', 'psi_dot'};
s2 = ss(A2, B2, C2, D2);
s2.StateName = {'phi', 'phi_dot'};
s2.InputName = {'Vl', 'Vr'};
s2.OutputName = {'phi', 'phi_dot'};
```

Первоначальные настройки проведены. Сохраняем данный Script под названием Rob_SM.m

Матрицы состояния можно посмотреть введением в рабочую строку матлаба команды s1 и s2.

Сборка Simulink модели робота-балансира

Создадим новую модель в Matlab Simulink. Открыв «Simulink library» добавим в нашу модель из вкладки Continuous блок «State-Space», из вкладки Commonly Used Blocks блок «Mux» и «Demux», а так же блок «Scope» и «Constant». Отрыв параметры блока «State-Space» впишем наши матрицы пространство состояния «A1», «B1», «C1», «D1»(Рис.18), а во второй блок с индексом 2. Соберем нашу модель, поставим в блоке «Constant» 1 и запустим моделирование. В блоке осциллографа видим, что графики уходят в бесконечность. Для проверки модели попробуем подвесить робота за колеса, по аналогии с маятником. Для этого изменим знак ускорения свободного падения. В скрипте parametrs_Rob.m запишем «-» перед 9,81 и запустим заного скрипт Rob_SM.m. После повторим моделирование.
В результате моделирования (Рис. 20) на графиках наблюдаем затухания колебаний. Модель робота-балансира работает адекватно.

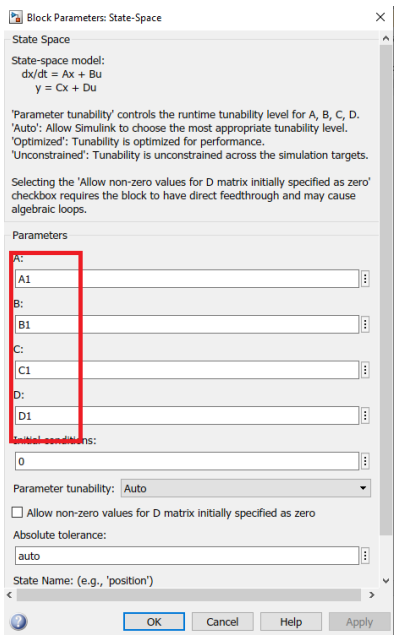


Рис.18 Настройка блока «State-Space»

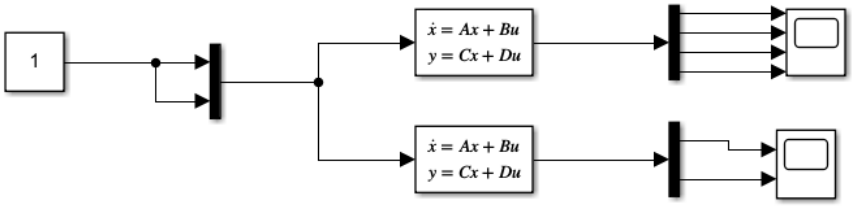


Рис.19. Модель робота-балансира

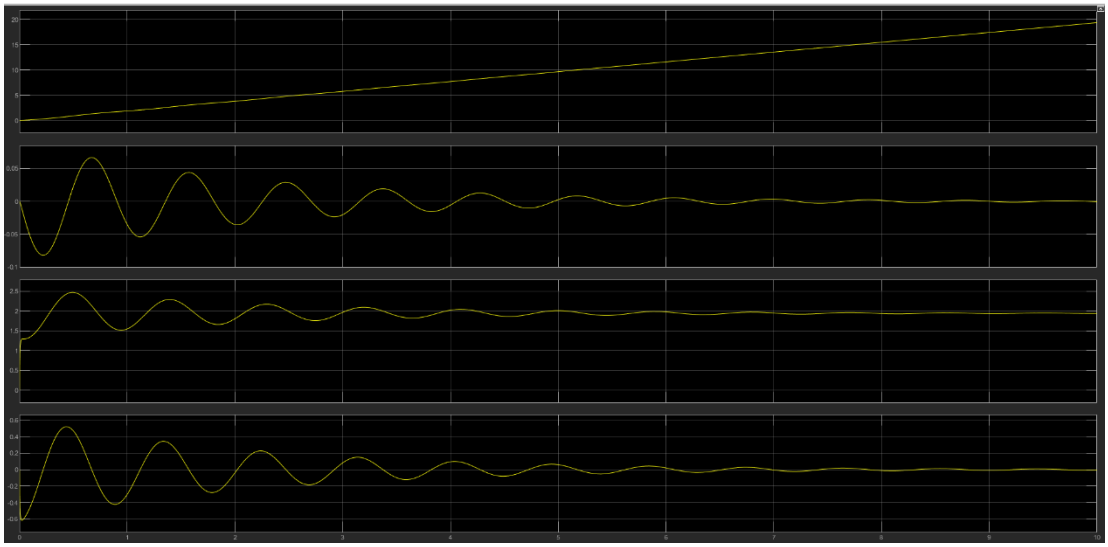


Рис.20. Результат моделирования робота-балансир

Варианты

вариант	вес колеса, [*кг*]	радиус колеса, [*м*]	масса робота, [*кг*]	ширина робота, [м]	Толщина робота, [м]	Высота робота, [м]	Коэффициент вязкого трения между колесом и полом	Коэффициент вязкого трения в моторе	Момент инерции ротора двигателя, [кгм^2]	Сопротивление обмотки двигателя, [Ом]	Коэффициент противо-ЭДС, [В*с/рад]	Коэффициент передачи по току, [Н*м/А]	Коэффициент передачи редуктора
1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	0,04	0,1	0,6	0,1	0,08	0,2	0	0,004	0,00001	6,9	0,4	0,5	1
2	0,02	0,1	0,9	0,1	0,04	0,3	0	0,003	0,00002	6,7	0,6	0,4	1
3	0,08	0,05	0,4	0,4	0,07	0,4	0	0,004	0,00002	6,4	0,4	0,6	3
4	0,03	0,1	0,5	0,1	0,08	0,4	0	0,002	0,00002	6,1	0,3	0,6	2
5	0,08	0,06	0,9	0,3	0,08	0,1	0	0,002	0,00001	7,2	0,7	0,4	3
6	0,08	0,07	0,7	0,3	0,06	0,4	0	0,003	0,00001	5,5	0,7	0,6	3
7	0,06	0,06	0,6	0,5	0,04	0,3	0	0,002	0,00001	6,9	0,4	0,3	1
8	0,04	0,02	0,5	0,3	0,06	0,4	0	0,002	0,00001	5,7	0,3	0,6	3
9	0,08	0,02	0,2	0,5	0,08	0,1	0	0,004	0,00001	8	0,7	0,4	3
10	0,06	0,08	0,4	0,4	0,08	0,1	0	0,004	0,00001	5,6	0,4	0,5	1
11	0,08	0,05	0,7	0,4	0,04	0,1	0	0,003	0,00002	7,7	0,4	0,5	2
12	0,09	0,06	0,5	0,1	0,08	0,2	0	0,002	0,00001	7,2	0,6	0,4	1
13	0,08	0,02	0,6	0,2	0,05	0,1	0	0,002	0,00001	6,1	0,4	0,6	2
14	0,06	0,07	0,5	0,4	0,06	0,3	0	0,002	0,00002	6	0,5	0,5	3
15	0,08	0,02	0,7	0,2	0,05	0,2	0	0,003	0,00001	5,4	0,5	0,6	2
16	0,09	0,04	0,7	0,1	0,07	0,4	0	0,003	0,00002	6	0,4	0,5	3
17	0,02	0,07	0,2	0,1	0,08	0,4	0	0,002	0,00002	6,8	0,5	0,5	2

1	2	3	4	5	6	7	0	9	10	11	12	13	14
18	0,03	0,09	0,8	0,2	0,07	0,3	0	0,004	0,00001	5,5	0,4	0,3	1
19	0,09	0,07	0,6	0,2	0,05	0,3	0	0,004	0,00002	5,6	0,5	0,3	3
20	0,02	0,1	0,6	0,2	0,07	0,2	0	0,003	0,00002	6,6	0,4	0,4	1
21	0,07	0,06	0,7	0,2	0,08	0,2	0	0,004	0,00001	6,4	0,7	0,5	1
22	0,1	0,08	0,7	0,1	0,06	0,3	0	0,003	0,00002	6,8	0,3	0,6	1
23	0,09	0,02	0,8	0,5	0,06	0,4	0	0,004	0,00001	6,2	0,3	0,4	1
24	0,09	0,02	0,6	0,1	0,05	0,1	0	0,002	0,00001	7,8	0,5	0,5	1
25	0,06	0,06	0,9	0,3	0,05	0,2	0	0,004	0,00001	8	0,4	0,5	3
26	0,06	0,07	0,8	0,4	0,07	0,1	0	0,004	0,00002	5,1	0,4	0,3	3
27	0,04	0,08	0,5	0,2	0,04	0,1	0	0,004	0,00002	6,9	0,7	0,3	2
28	0,09	0,07	0,4	0,5	0,05	0,1	0	0,004	0,00001	8	0,5	0,4	2

Контрольные вопросы

1. Что такое Маятник?
2. Что такое колебательное движение?
3. Что называют свободным колебанием?
4. Что такое автоколебания?
5. Что такое метод пространство состояния?
6. Второй закон Ньютона для вращающегося теля
7. Вектор управления
8. Вектор выхода

Системная модель робота

Для данного проекта системная модель будет связующем элементом между моделью робота и системы управления. Начнем с анализа робота. Для поддержания баланса требуется знать угол отклонения от вертикали и углы поворотов двигателей. В роботе установлены электродвигатели постоянного тока с энкодерами и гироскоп. Управление скоростью вращения двигателей происходит путем изменения скважности ШИМ.

Скважность ШИМ	Коэффициент энкодера	Коэффициент гироскопа
-127:127	1^0	$1^0/c$

Модель робота принимает на вход значения напряжения, и выдает состояние системы. Структорная схема содержит следующие компоненты:
На вход в систему поступает ШИМ сигнал, который преобразуется в значение напряжения. Напряжение поступает на модель робота-балансира, а выходом будут показания с датчика наклона(Гироскоп) и углов поворота двигателей(энкодеров) (Рис.21)

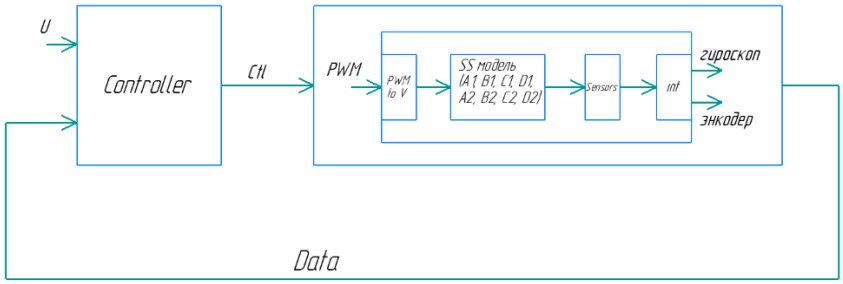


Рис.21 Структурная модель.

К данной модели добавим заготовочную модель контроллера, информационные шины и добавим управление.
Добавим блок «Gain» и запишем в него 1/K_PWM. Этот блок будет отвечать за перевод значений в вольты. Для квантования по уровню добавим «Round». Для уменьшения элементов на модели робота свернем часть модели, выбирим элементы и нажмем на «Create Subsystem from select»(Рис.22). С целью учета ШИМ добавим блок «Saturation» и укажем пределы -127:127.

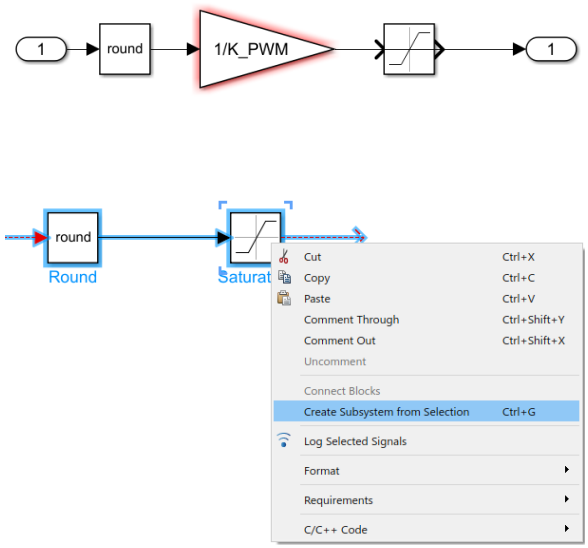


Рис.22 Пример сворачивания части модели

С целью работы с идеальным объектом можно опускать не идеальности. Для этого предусмотрим в модели возможность отключения не идеальности. Нажмем правой кнопкой в созданную подсистему и сделаем ее вариантной, путем нажатия «Variant Subsystem».(Рис.23)

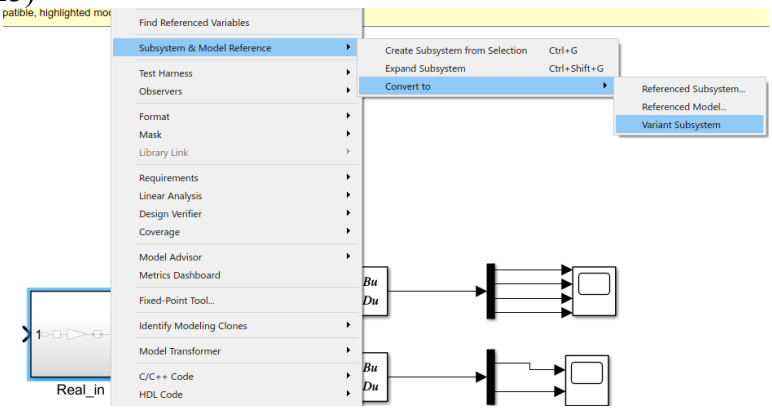


Рис.23 Работа с подмоделью

Внутри подсистемы создадим копию Real_in, назовем его Ideal_in и удалим из нее блоки не идеальности(Round, Saturation). Переключатся будем программно. Для этого в параметрах подсистемы (Block Parameters) Ideal_in сделаем вариантом по умолчанию(default), а для варианта Real_in добавим условие real_model == 1 (Рис. 25)

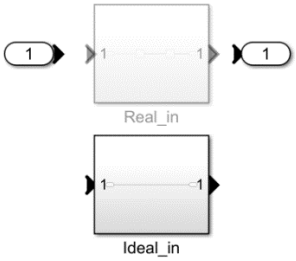


Рис. 24. Идеальная и реальная модель

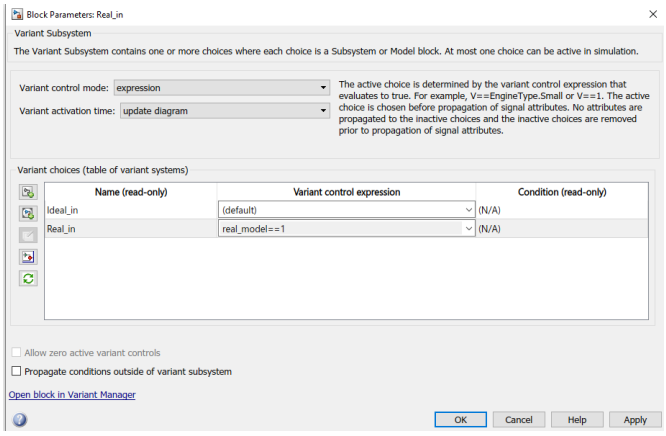


Рис. 25 Блок параметров подсистемы

Создадим новый скрипт и запишем real_model=0;. Сохраним под названием config.m.
В файле parametrs_Rob.m добавим

K_PWM= $\frac{127}{U_{\text{пит}}} = \frac{127}{12} = 10{,}58$

Настроим работу с датчиками. Возьмём уравнение 6:

$$\theta = \frac{1}{2}(\theta_l + \theta_r)$$
$$\phi = \frac{R}{W}(\theta_l - \theta_r)$$

Добавим к ним следующее уравнение:

$$\Theta_{ml,r} = \theta_{l,r} - \psi$$

Из данных уравнение выводим необходимые формулы:

$$enc_l = \theta_{ml} = \theta - \frac{W}{2R}\phi - \psi$$
$$enc_r = \theta_{mr} = \theta + \frac{W}{2R}\phi - \psi$$
$$gyro = \dot{\psi}$$

Для подстановки в нашу модель заменим переменные на компоненты вектора y системы и переведем из градусов в радианы:

$$y = \begin{bmatrix} y_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} \theta \\ \psi \\ \dot{\theta} \\ \dot{\psi} \\ \phi \\ \dot{\phi} \end{bmatrix} \rightarrow \begin{matrix} enc_l = y_1 - \frac{W}{2R}y_5 - y_2 \\ enc_r = y_1 + \frac{W}{2R}y_5 - y_2 \\ gyro = y_4 \end{matrix} \quad 37$$

Данное выражение запишем в блок «Matlab Function» во вкладке User-Defined Functions. Внутрь блока запишем 37 в языке матлаб:

```
function [enc_l,enc_r,gyro]=fcn(y,R, W)
enc_l=rad2deg(y(1) - W/R/2 * y (5) - y(2));
enc_r=rad2deg(y(1) + W/R/2 * y(5) - y(2));
gyro=rad2deg(y(4));
```

Данная функция выдает в результате своей работы данные с энкодеров и гироскопа, а качестве аргумента получает вектор выхода y, радиус колеса R и ширину робота W.

Следующим шагом настроим информационную шину данных. Для этого в модели Simulink зайдем в Model Explorer и поменяем параметры в блоке Matlab function укажем, что R и W являются параметрами (Рис.26).

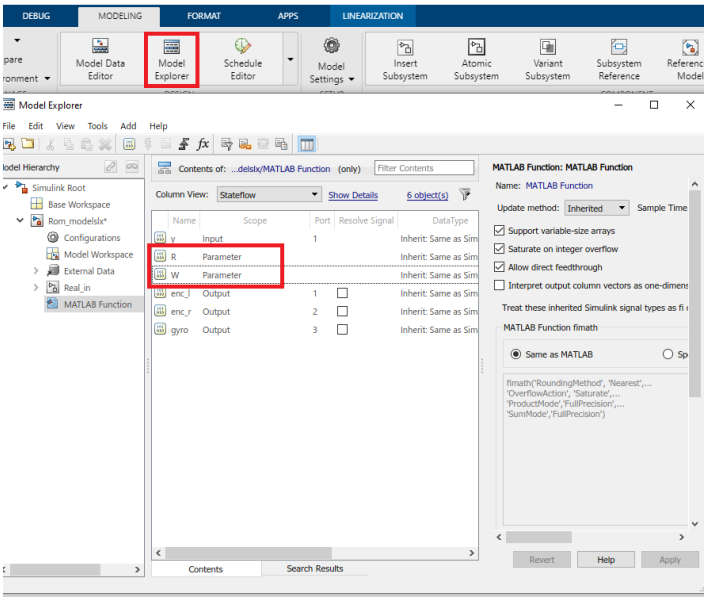


Рис.26. Настройки Model Explorer

Выходы с блока «Matlab Function» объединим в вектор при помощи блока Mux и скопируем нашу подсистему Real_in и переименуем скопированную подсистему в Real_out. Из подсистемы Real_out уберем блок Ideal_in, а блок Real_in переименуем в Real_out и убираем насыщение (рис. 27). Свернем все в подсистему Plant.

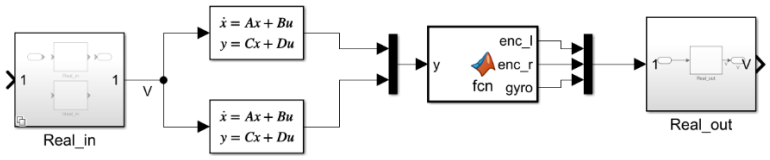


Рис.27 Подсистема Plant

Из библиотеки Library Browser вставим новую подсистему (блок «Subsystem») и назовем ее «Controller». Внутри подсистемы «Controller» добавим новый вход и переименуем входы в U и Data, а выход в Ctl. Таким образом контроллер на вход принимает управляющий сигнал U и сигналы с датчиков Data, а на выходе формирует управляющее воздействие Ctl (Рис. 28).



Рис.28 Подсистема «Controller»

Соберем нашу системную модель:

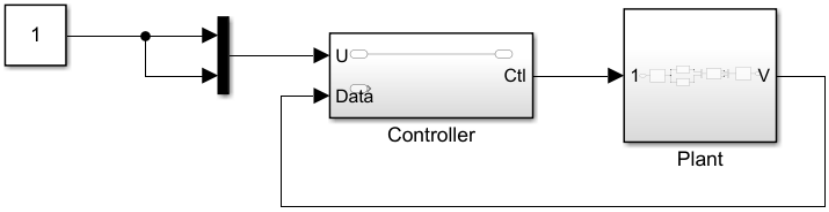


Рис.29 Системная модель

Следующим шагом организуем передачу данных между подсистемами. Для этого в меню «Edit» откроем инструмент «Bus Editor» и создадим новую шину нажав соответственно Add Bus (Рис. 30)

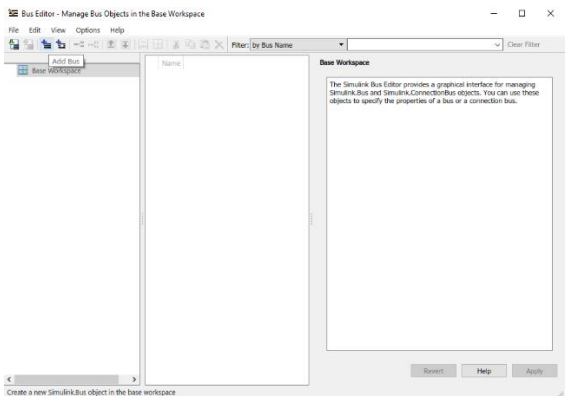


Рис. 30 Bus editor

Назовем его Ctl и создадим 1 сигнал PWM нажав соответственно Add/Insert Bus element и назначим число элементов равных двум, соответственно скорость левого и правого колеса(Рис.31)

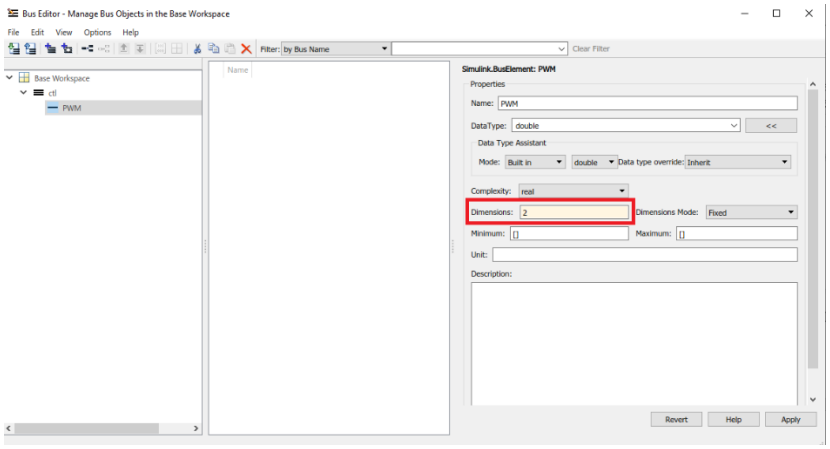


Рис.31 Настройки шины PWM

Создадим еще шину Data, нажав соответственно Add Bus и включим в него 2 элемента нажав Add/Insert.(Рис.32)

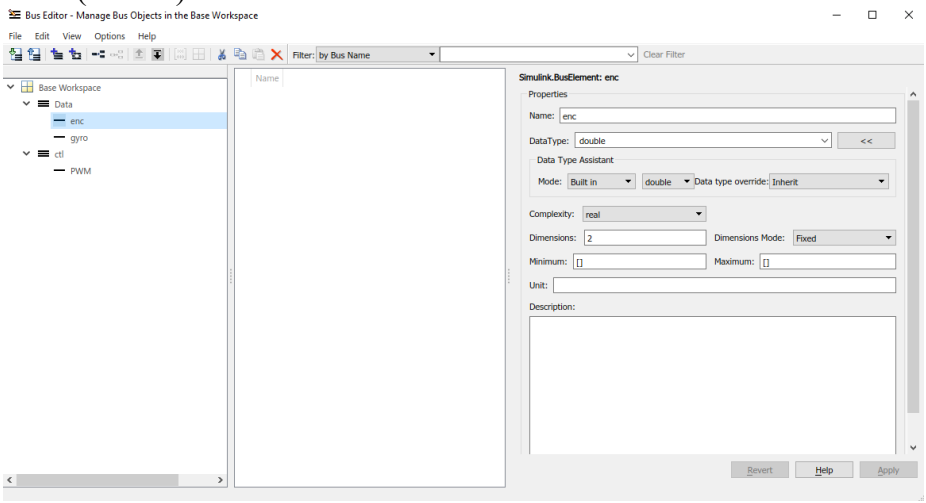


Рис.32 Bus editor после внесения в него шин данных

Чтобы не потерять сохраним шину с настройками в .mat файл нажав File-export to file. Добавим блок Mux, Demux и Bus creator в подсистему Plant. Подписываем сигналы. В настройках Bus Creator укажем тип Data (Рис.33).

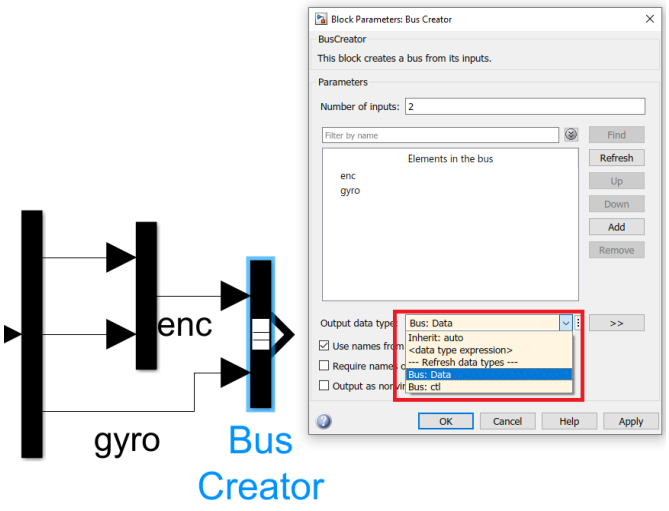


Рис. 33. Настройки Bus Cretor

Откроем подсистему «Controller» добавим блок «Bus Selector» и блок «Block Creator». В настройках блока «Bus Selector» добавим сигналы с датчиков выбрав сигналы слева и нажав Select:

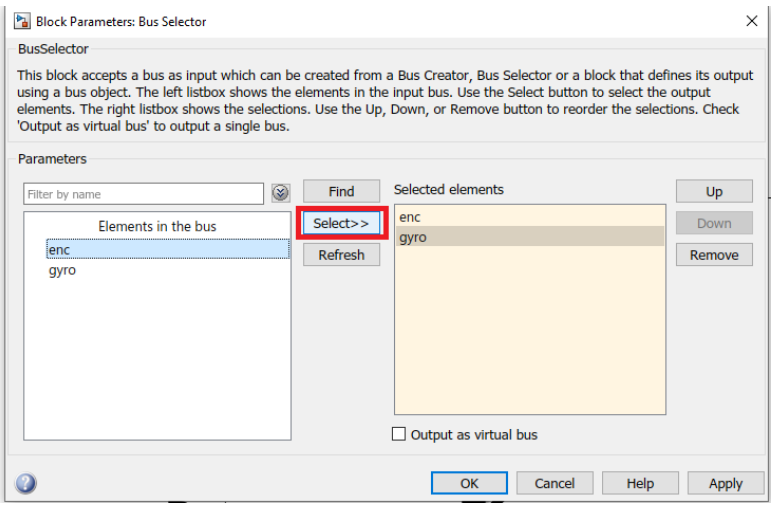


Рис.34 Настройки блока Bus Selector

Блок «Bus Creator» добавим в связь с напряжением и настроем в нем данные типа Ctl(Рис. 35). После этого добавим блок Bus selector в подсистеме Plant и укажем сигнал PWM.

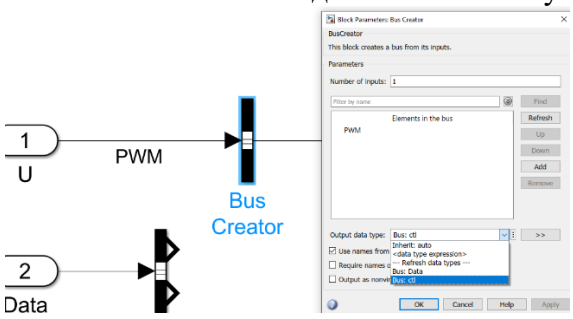


Рис.35 Настройки Bus Creator в подсистеме Controller

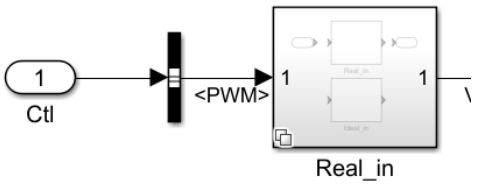
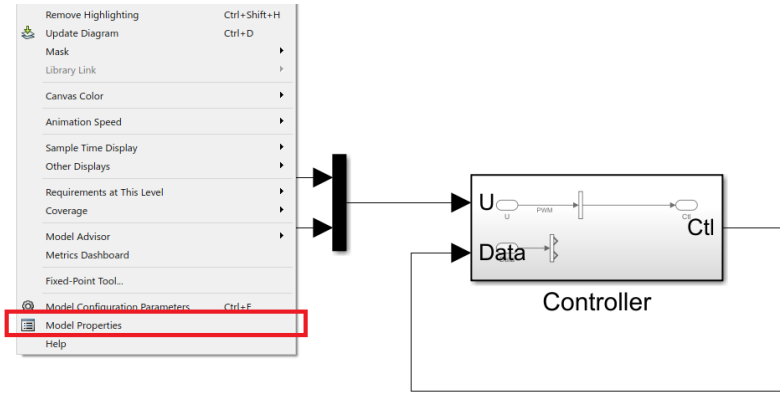


Рис.35 Настройки Bus Creator в подсистеме Plant

С целью автоматизации загрузки настроек информационных шин создадим Script, который будет загружать настройки из ранее сохраненного .mat файла.
Создаем новый Script и запишем туда команду:
load('bus_data.mat')
Сохраним его назначив имя preload.m
Чтобы каждый раз вручную не запускать требуемые Script ы, назначим их в Simulink модели. Для этого кликнем правой кнопкой на пустом месте Simulink модели и нажмем Model Properties:



В открывшимся меню зайдем во вкладку «Callbacks» и в пункте PreloadFcn напишем имена Script ов preload и config (Рис 36).

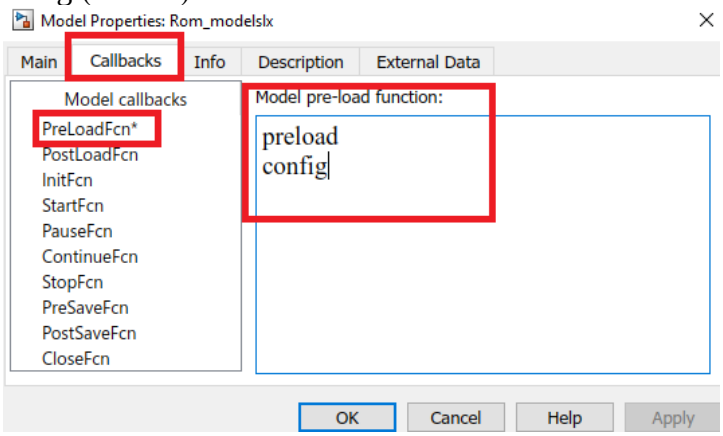


Рис. 36. Настройка запуска скриптов перед выполнением моделирования

В пункте InitFcn пропишем имя скрипта Rob_SM

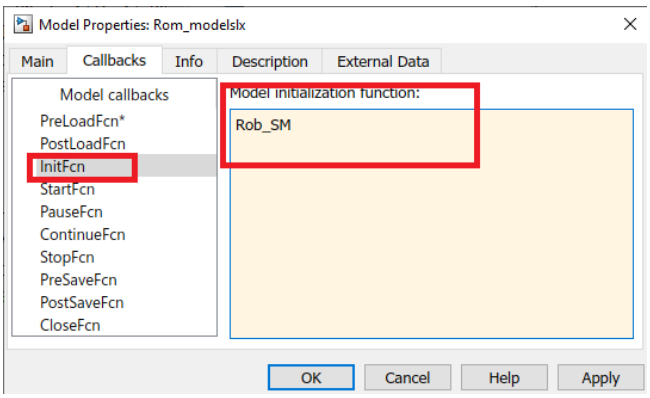


Рис. 37. Настройка инициализации

Зайдем в подсистему Plant и добавим осциллографы для наблюдения:

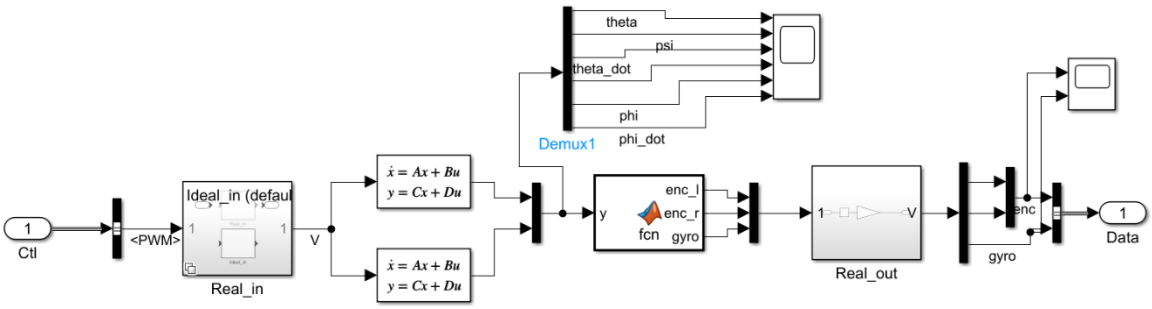


Рис.38. Подсистем Plant

С отрицательным ускорением свободного падения после запуска моделирования получаем модель маятника:

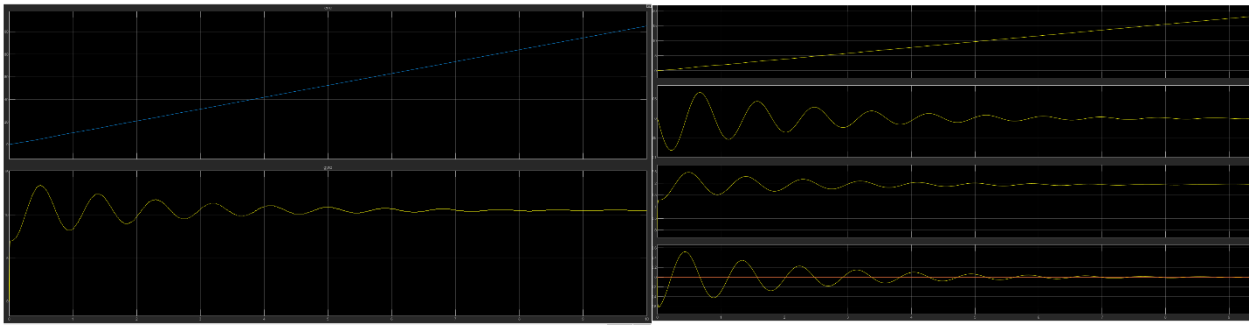


Рис. 39. Результаты моделирования

Если в параметрах config поставим 1, то после моделирования увидим все не идеальности работы нашей модели:

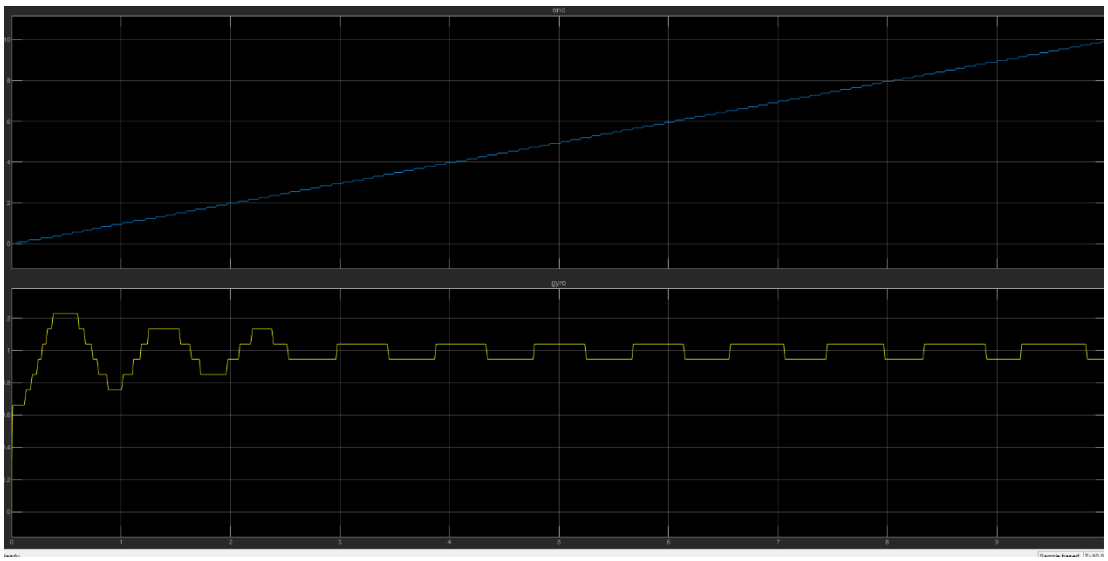


Рис. 39. Результаты моделирования с неидеальными датчиками

Разработка системы управления

Основными характеристиками системы управления являются устойчивость и качество регулирования. Робот контроллер будет стоять только в том случае, если разработан контроллер, который обеспечит устойчивость всей системы.

Качество системы управления выражается конкретными характеристиками переходного процесса при ступенчатом управляющем воздействии: Время переходного процесса, перерегулирование и статическая ошибка. Для данной системы имеет место использования линейно-квадратичного регулятора.

Линейно-квадратичный регулятор (англ. *Linear quadratic regulator*, *LQR*) — в теории управления один из видов оптимальных регуляторов, использующий квадратичный функционал качества. Задача, в которой динамическая система описывается линейными дифференциальными уравнениями, а показатель качества представляет собой квадратичный функционал, называется задачей линейно-квадратичного управления. Широкое распространение получили линейно-квадратичные регуляторы (LQR). Задачей регулятора является обеспечение оптимального управления согласно квадратичному критерию качества.

Для непрерывных линейных систем, описываемых в пространстве состояний системой уравнений

$$\dot{x} = A(t)x + B(t)u$$

Опишу потом

С целью стабилизации робота зададим роботу его состояние. Возьмём ранее выведенные уравнения

$$\theta = \frac{1}{2}(\theta_l + \theta_r)$$

$$\Theta_{ml,r} = \theta_{l,r} - \psi$$

Из данных уравнений выведем следующие соотношения для переменных состояний:

$$\theta = \frac{1}{2}(\theta_{ml} + \theta_{mr}) + \psi$$

$$\psi = \int \dot{\psi} dt$$

$$\dot{\theta} = \frac{d}{dt}\theta$$

$$\dot{\psi} = \dot{\psi}$$

Опишем все это в нашей Simulink модели:

Первым делом уберем знак минуса перед g. Для начала будем работать с идеализированной моделью. Откроем в Simulink моделе и в подсистеме «Controller» добавим блок «Degrees to Radians» (блок перевода градусов в радианы), блок сумматора «Sum of Elements», «Gain», «integrator», «Add», «Derivative» и «bus Creator»(Рис. 40)

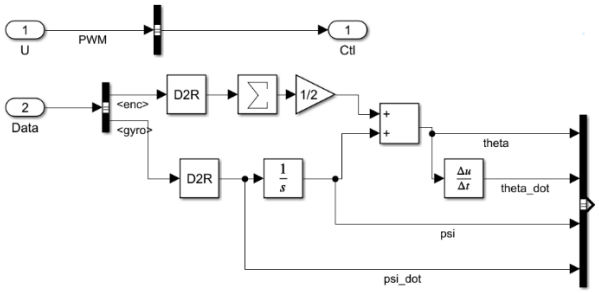


Рис. 40. Подсистема «Control»

Часть подсистемы «Control» так же свернем в подсистему и назовем его «Get_states»:

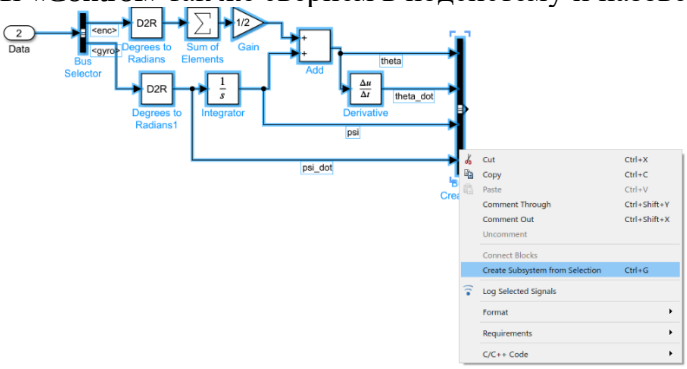


Рис.41 Организации подсистемы «Get_states»

Добавим в подсистему блок «Bus Selectro» и организуем передачу данных следующим порядком: theta, psi, theta_dot, psi_dot и преобразуем выход в 1 вектор поставив галочку в пункте «Output as bus»(Рис. 42). Добавим блок «Bus to vector» и включим его в модель. Выход блока «Bus to vector» и есть выход состояния X.

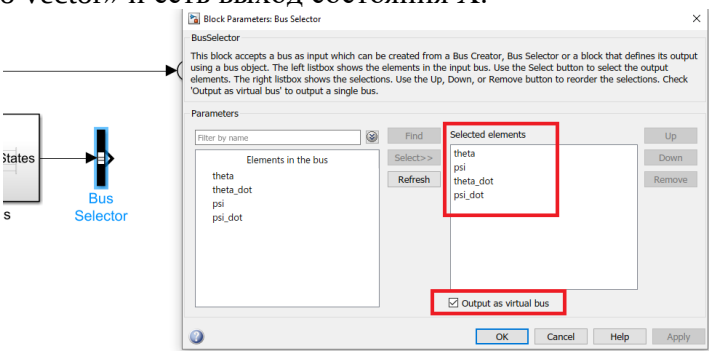


Рис.42. Настройки Bus Selector

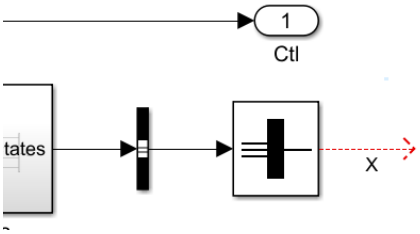


Рис. 43. Блок «Bus to vector»

Опишем сам регулятор в Matlab Script. Создадим новый скрипт. Назовем его «control» и в нем опишем LQR регулятор:

```
QQ=eye(4);
RR=eye(2);
Klqr=lqr(s1,QQ,RR);
```

Первая строка создает единичную матрицу размерностью 4, вторая размерностью 2, а третья строка синтезирует lqr контроллер. Принимает в качестве аргументов матрицу пространство состояния робота s1, а так же две единичные матрицы. Коэффициенты контроллера можно посмотреть введя в командную строку Klqr:

```
>> Klqr

Klqr =

   -0.7071  -39.4600   -1.5731   -4.0596
   -0.7071  -39.4600   -1.5731   -4.0596
```

Пропишем скрипт control в функции инициализации(Рис.37). Начнем организации отрицательной связи. Уставку будем задавать сами, а вычитать будем показания с датчиков. Добавим в подсистему «Control» сумматор блок «Sum», «Gain», «Mux». Откроем настройки блока «Gain» и укажем матричное преумножение и укажем матрицу коэффициентов K(Рис.44).

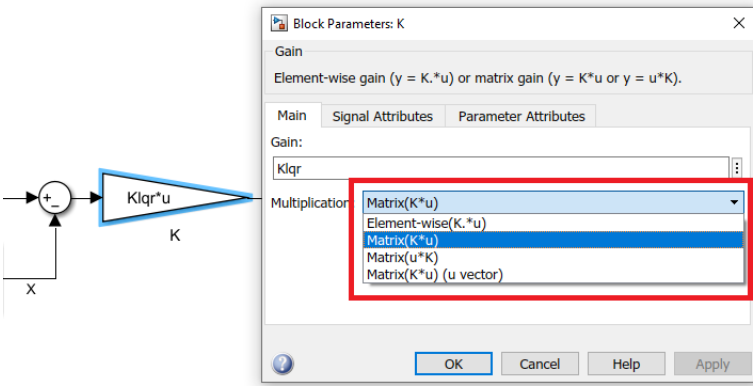


Рис. 44. Настройка блока «Gain»

На вход сумматора приходят желаемые значения вектора уставки, сравнивается с данными с датчиков, вычисляется ошибка регулирования, после до умножается на коэффициенты регулятора и на коэффициент ШИМ и на выходе получим управляющее воздействие

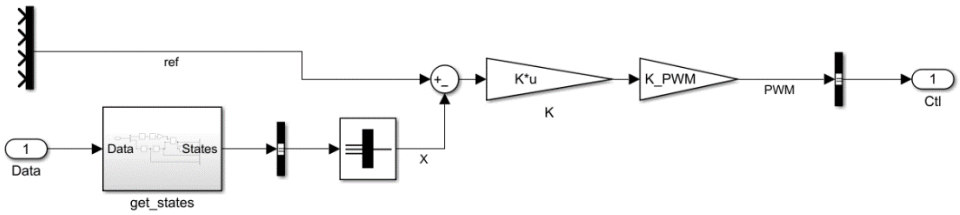


Рис. 45. Подсистема «Control»

Для проверки системы управления зададим в подсистеме «Plant» в блоке «State-Space» начальное состояние [0 Psi0 0 0].

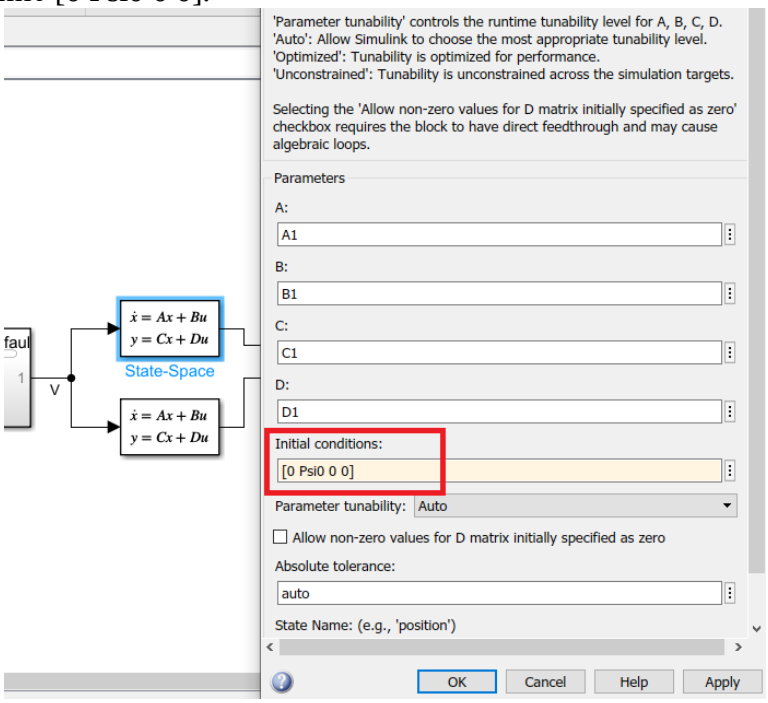


Рис.46 Начальные условия

Psi0-это начальный наклон робота. Мы можем его прописывать в командной строке, но легче прописать в скрипте параметров робота parametrs_Rob.m. Назначим начальное положение равное 0.01-0.05 радиана

```
n=1;                                    % Коэффициент передачи редуктс
K_PWM=10.58;                            %Параметры ШИМ
Psi0=0.01;                              % Начальный наклон робота
```

Запустим моделирование нажав на кнопку Run в Simulink модели и посмотрим результаты моделирования на осциллографе. (Рис. 47).

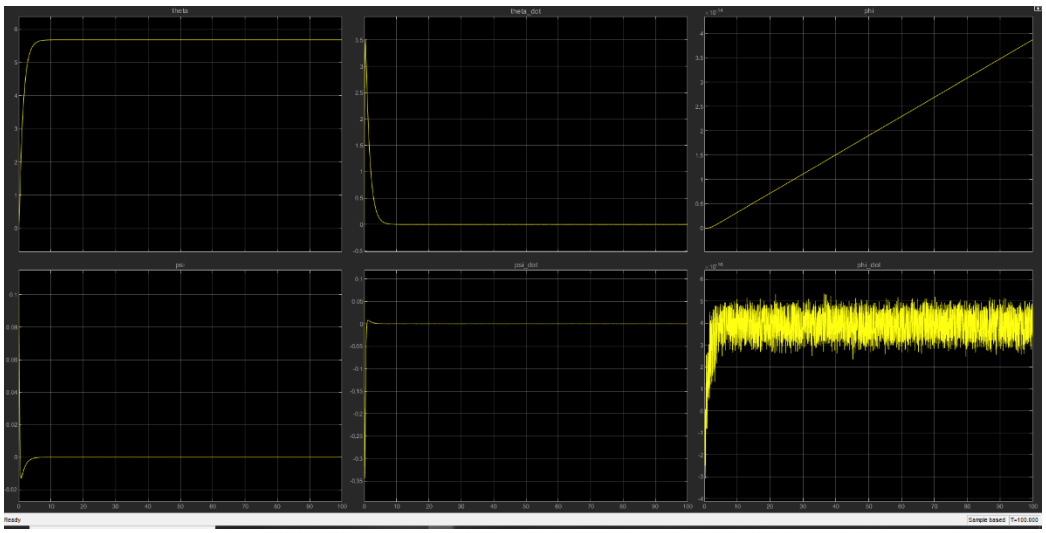


Рис.47. Результаты моделирования

Из анализа графиков можно сделать вывод о том, что робот стремиться к устойчивому положение, но присутствует статическая ошибка по углу. Устраним его включив интегрирование в регулятор.

Опишем интегратор в модели робота пространство состояния. Откроем скрипт Rob_SM.m и добавим:

```
s0=ss(1/tf('s'));
s0.StateName='theta_int';
s0.OutputName='theta_int';
s3=append(s0,s1);
s3.A(1,2)=1;
s3(:,1)=[];
```

```
30  C2 = eye(2);
31  D2 = zeros(2);
32  % Модель пространство состояния
33  s1 = ss(A1, B1, C1, D1);
34  s1.StateName = {'theta', 'psi', 'theta_dot', 'psi_dot'};
35  s1.InputName = {'Vl', 'Vr'};
36  s1.OutputName = {'theta', 'psi', 'theta_dot', 'psi_dot'};
37  s2 = ss(A2, B2, C2, D2);
38  s2.StateName = {'phi', 'phi_dot'};
39  s2.InputName = {'Vl', 'Vr'};
40  s2.OutputName = {'phi', 'phi_dot'};
41  s0=ss(1/tf('s'));
42  s0.StateName='theta_int';
43  s0.OutputName='theta_int';
44  s3=append(s0,s1);
45  s3.A(1,2)=1;
46  s3(:,1)=[];
```

Рис.48. Добавляем интегратор

После добавления интегратора поменяем настройки LQR регулятора. Зададим размерность матрицы QQ равным 5, и сам регулятор будем синтезировать с матрицы s3

```
Klqr=lqr(s3,QQ,RR);
```

Внесем поправки в Simulink модели. Первым делом добавим интегратор в подсистеме «get_states», он будет интегрировать угол theta. В настройках Bus Selector а поставим theta_int на первое место.

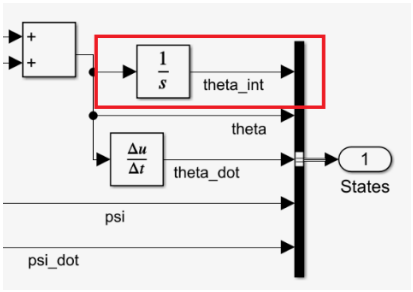


Рис. 49. Добавляем интегратор__

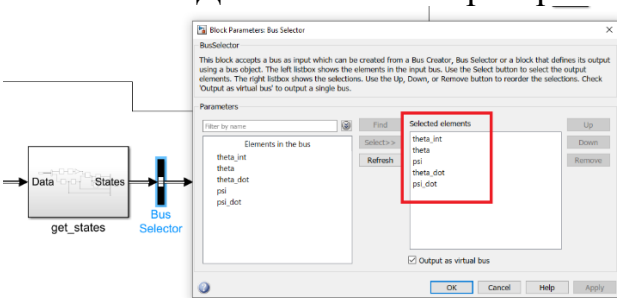


Рис. 50. Настройки Bus Selector.

В задающем векторе добавим 5 элемент и пока что оставим его равным нулю. Запустим моделирование. Из графиков видно, что статическая ошибка стремится к нулю.

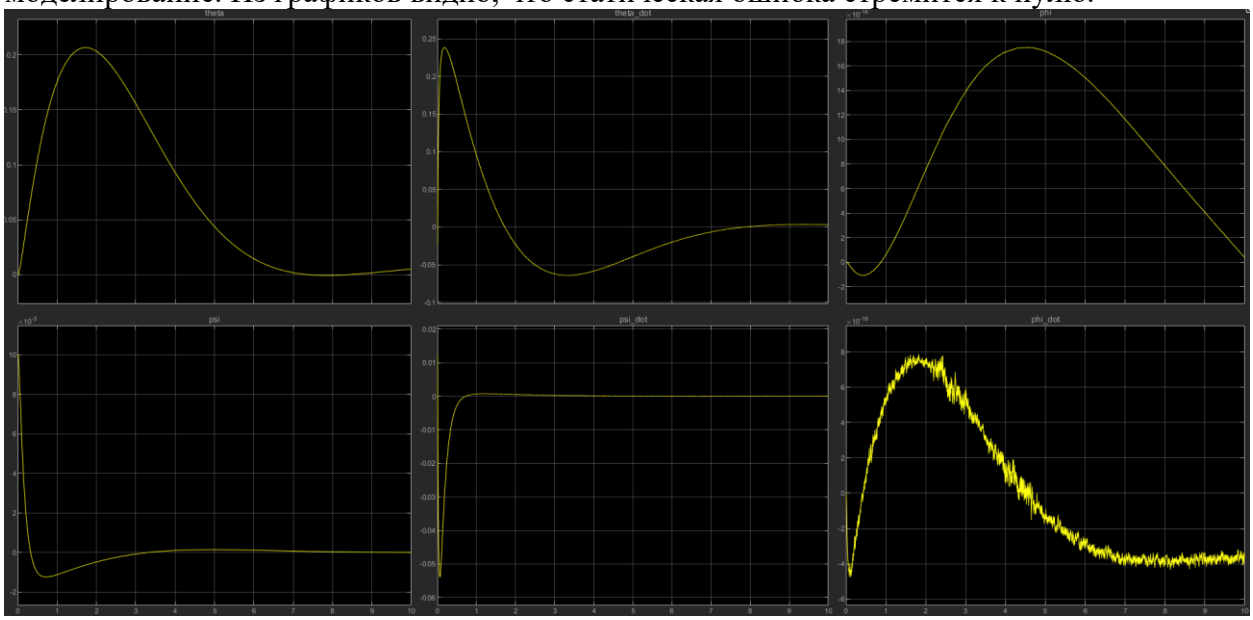


Рис. 51 Результаты моделирования с интегратором

Добавим возможность управления роботом. Для этого создадим новую систему s4, которая будет объединять систему s3 и s2 добавив следующие строки в скрипте Rob_SM.m:

```
s4=append(s3,s2);
s4.B(end, [1,2])=s4.B(end,[3,4]);
s4(:,[3,4])=[];
```

Перенастроим LQR контроллер в скрипте control.m следующим образом:

```
QQ=eye(7);
RR=eye(2);
```

Klqr=lqr(s4,QQ,RR);

После этого добавим следующие изменения в Simulink модели. В подсистеме get_states добавим сумматор, блок усилителя «Gain», блок «Demux» и блок Derivative. Соединим все согласно формуле:

$$\phi = \frac{R}{W}(\theta_r - \theta_l)$$

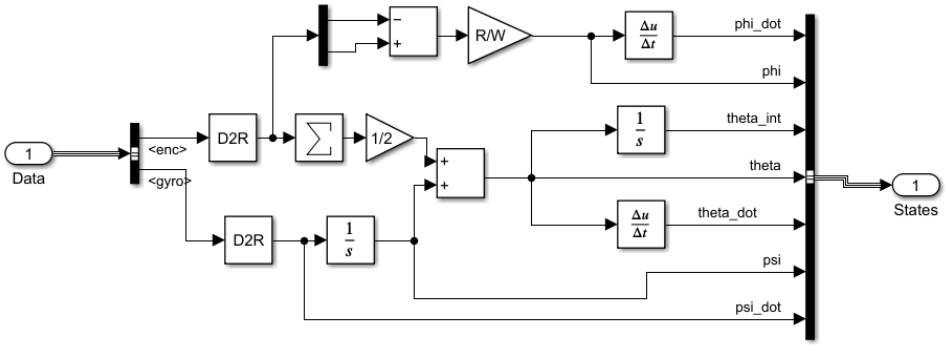


Рис. 52. Подсистема get_states

Настроем блок Bus Selector поставив ϕ и ϕ̇ в конец шины.

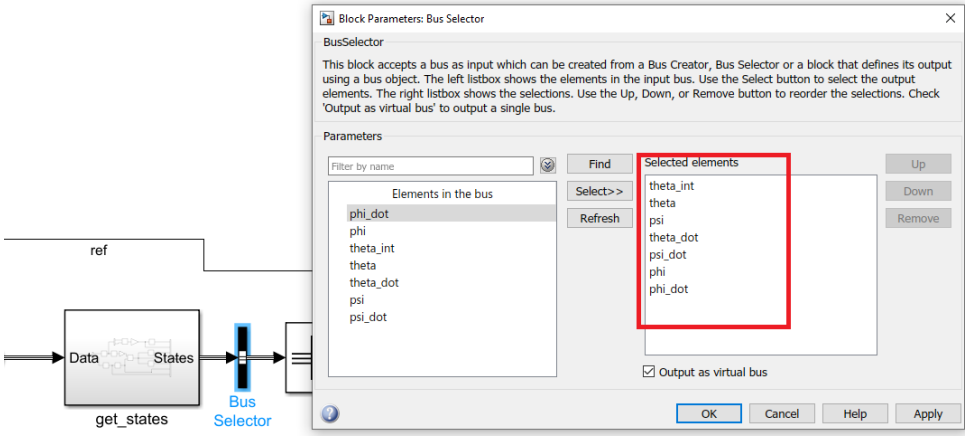


Рис. 50. Настройки Bus Selector.

К задающему вектору добавим еще 2 входа. Зададим скорости 1 рад/с, углы отклонения робота зададим равными нулю (иначе робот упадет) и свернем все в подсистему. И произведем моделирование.

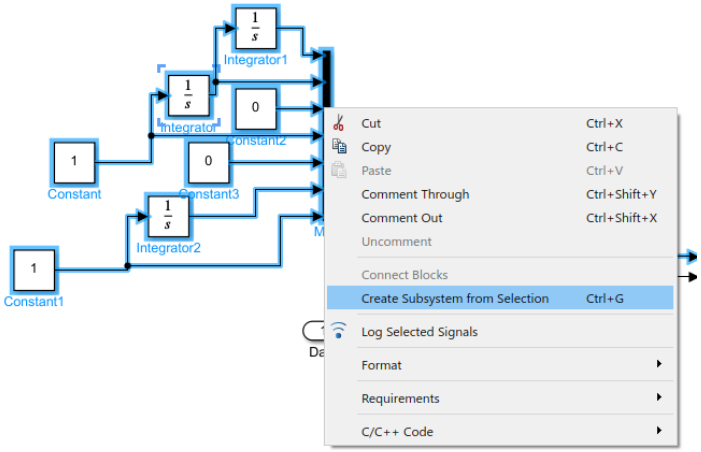


Рис. 51. Задаем управляющие воздействия

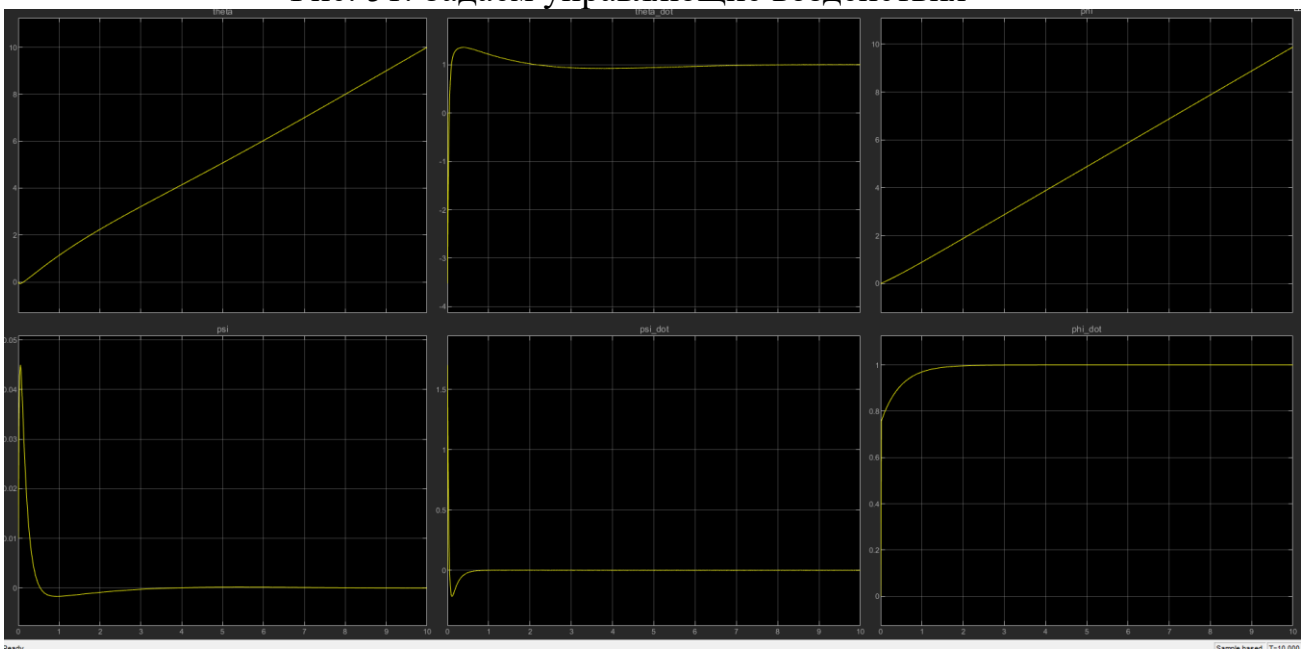


Рис. 52 Результаты моделирования.

Из анализа результатов моделирования видим, что углы отклонения робота равны нулю, значит робот устойчив. Скорости движения робота равны одному, как мы и задали. Значения с энкодеров возрастают, значит робот едет.

Промоделируем с неидеальными датчиками. Для этого в скрипте config.m изменим значение real_model=1 и запустим моделирование.

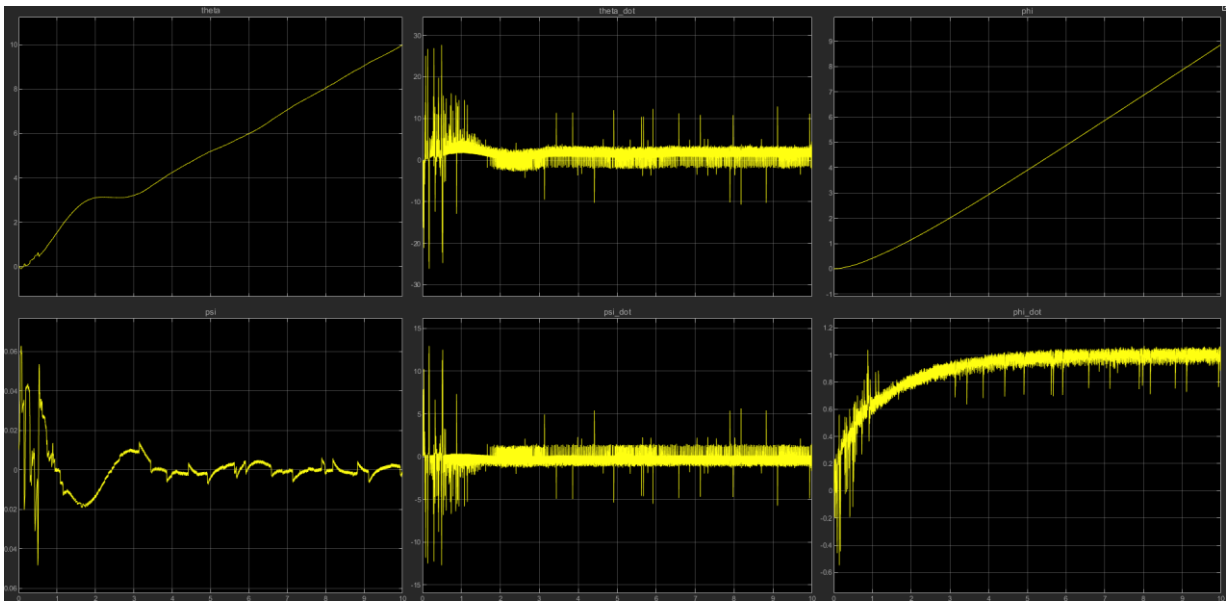


Рис. 52 Результаты моделирования с неидеальными датчиками

Приложение

Исходные данные